

LAPORAN
TUGAS PRAKTIKUM 3 – SISTEM OPERASI



Dosen pengampu
Triawan Adi Cahyanto M.Kom

Disusun Oleh :
AZQI FIRMAN M.(2410651064)

Tugas 1: Kalkulator Paralel

```
GNU nano 7.2      kalkulator_paralel.c
// kalkulator_paralel.c
// Compile: gcc -pthread -o kalkulator_paralel kalkulator_paralel.c
// Run: ./kalkulator_paralel

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

typedef struct {
    double a;
    double b;
    double result;
    int error; // 0 = ok, 1 = error (e.g., div by zero)
} op_args_t;

void *thread_add(void *arg) {
    op_args_t *p = (op_args_t*)arg;
    p->result = p->a + p->b;
    p->error = 0;
    return NULL;
}

void *thread_sub(void *arg) {
    op_args_t *p = (op_args_t*)arg;
    p->result = p->a - p->b;
    p->error = 0;
    return NULL;
}

void *thread_mul(void *arg) {
    op_args_t *p = (op_args_t*)arg;
    p->result = p->a * p->b;
    p->error = 0;
    return NULL;
}

void *thread_div(void *arg) {
    op_args_t *p = (op_args_t*)arg;
    if (p->b == 0.0) {
```

```
azqi@MSI: ~  
GNU nano 7.2 kalkulator_paralel.c  
    p->error = 1; // division by zero  
} else {  
    p->result = p->a / p->b;  
    p->error = 0;  
}  
return NULL;  
}  
  
int main(void) {  
    double a, b;  
    printf("Masukkan 2 angka (pisahkan dengan spasi): ");  
    if (scanf("%lf %lf", &a, &b) != 2) {  
        fprintf(stderr, "Input tidak valid\n");  
        return 1;  
    }  
  
    op_args_t add_args = {a, b, 0.0, 0};  
    op_args_t sub_args = {a, b, 0.0, 0};  
    op_args_t mul_args = {a, b, 0.0, 0};  
    op_args_t div_args = {a, b, 0.0, 0};  
  
    pthread_t tid_add, tid_sub, tid_mul, tid_div;  
  
    // Create threads  
    if (pthread_create(&tid_add, NULL, thread_add, &add_args) != 0) {  
        perror("pthread_create add");  
        return 1;  
    }  
    if (pthread_create(&tid_sub, NULL, thread_sub, &sub_args) != 0) {  
        perror("pthread_create sub");  
        return 1;  
    }  
    if (pthread_create(&tid_mul, NULL, thread_mul, &mul_args) != 0) {  
        perror("pthread_create mul");  
        return 1;  
    }  
    if (pthread_create(&tid_div, NULL, thread_div, &div_args) != 0) {  
        perror("pthread_create div");  
        return 1;  
    }  
}
```

```
}  
  
// Wait for all threads to finish  
pthread_join(tid_add, NULL);  
pthread_join(tid_sub, NULL);  
pthread_join(tid_mul, NULL);  
pthread_join(tid_div, NULL);  
  
// Tampilkan hasil setelah semua selesai  
printf("\nHasil operasi (setelah semua thread selesai):\n");  
printf("Penjumlahan : %.10g\n", add_args.result);  
printf("Pengurangan : %.10g\n", sub_args.result);  
printf("Perkalian : %.10g\n", mul_args.result);  
if (div_args.error) {  
    printf("Pembagian : error (pembagian dengan nol)\n");  
} else {  
    printf("Pembagian : %.10g\n", div_args.result);  
}  
  
return 0;
```


Program ini adalah **kalkulator paralel** yang menghitung **penjumlahan, pengurangan, perkalian, dan pembagian** dari dua angka **secara bersamaan** menggunakan **thread**.

Langkah kerjanya:

1. Program minta user memasukkan **dua angka** (a dan b).
2. Untuk setiap operasi matematika dibuat **thread terpisah**:
 - o `thread_add` → penjumlahan
 - o `thread_sub` → pengurangan
 - o `thread_mul` → perkalian
 - o `thread_div` → pembagian (cek pembagian dengan nol)
3. Semua thread dijalankan **bersamaan**, jadi perhitungan bisa paralel.
4. Program menunggu semua thread selesai (`pthread_join`).
5. Setelah selesai, program menampilkan hasil semua operasi sekaligus.
6. Jika user memasukkan $b = 0$, thread pembagian akan menandai **error**, sehingga ditampilkan pesan pembagian dengan nol.

Intinya: **setiap operasi matematik dilakukan di thread berbeda agar bisa dihitung paralel**, lalu hasilnya ditampilkan setelah semua thread selesai.

OUTPUT:

```
azqi@MSI:~$ ./kalkulator_paralel
Masukkan 2 angka (pisahkan dengan spasi): 12 4

Hasil operasi (setelah semua thread selesai):
Penjumlahan : 16
Pengurangan : 8
Perkalian   : 48
Pembagian   : 3
azqi@MSI:~$ ./kalkulator_paralel
Masukkan 2 angka (pisahkan dengan spasi): 11 7

Hasil operasi (setelah semua thread selesai):
Penjumlahan : 18
Pengurangan : 4
Perkalian   : 77
Pembagian   : 1.571428571
```

Tugas 2: File Processing

```
azqi@MSI: ~  
GNU nano 7.2 file_processing.c  
// file_processing.c  
// Compile: gcc -pthread -o file_processing file_processing.c  
// Run: ./file_processing nama_file.txt  
  
#include <stdio.h>  
#include <stdlib.h>  
#include <pthread.h>  
#include <string.h>  
#include <ctype.h>  
#include <time.h>  
  
// Struktur argumen untuk setiap thread  
typedef struct {  
    char *text;  
    long start;  
    long end;  
    long count;  
} ThreadArgs;  
  
// Fungsi untuk menghitung jumlah kata dalam range teks  
long count_words_range(const char *text, long start, long end) {  
    long count = 0;  
    int in_word = 0;  
  
    for (long i = start; i < end; i++) {  
        if (isspace(text[i]) || text[i] == '\\0') {  
            in_word = 0;  
        } else if (!in_word) {  
            in_word = 1;  
            count++;  
        }  
    }  
    return count;  
}  
  
// Fungsi yang dijalankan oleh thread  
void *thread_count(void *arg) {  
    ThreadArgs *t = (ThreadArgs *)arg;  
    t->count = count_words_range(t->text, t->start, t->end);  
}
```

[Read 114 lines]


```

    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <nama_file.txt>\n", argv[0]);
        return 1;
    }

    // Membuka file input
    FILE *f = fopen(argv[1], "r");
    if (!f) {
        perror("Tidak bisa membuka file");
        return 1;
    }

    // Membaca isi file ke memory
    fseek(f, 0, SEEK_END);
    long size = ftell(f);
    fseek(f, 0, SEEK_SET);

    char *text = malloc(size + 1);
    fread(text, 1, size, f);
    text[size] = '\0';
    fclose(f);

    // --- SINGLE THREAD ---
    clock_t start_single = clock();
    long total_single = count_words_range(text, 0, size);
    clock_t end_single = clock();
    double time_single = (double)(end_single - start_single) / CLOCKS_PER_SEC;

    // --- MULTITHREAD ---
    pthread_t t1, t2;
    ThreadArgs a1 = {text, 0, size / 2, 0};
    ThreadArgs a2 = {text, size / 2, size, 0};

    clock_t start_multi = clock();
    pthread_create(&t1, NULL, thread_count, &a1);

```

```

clock_t start_multi = clock();
pthread_create(&t1, NULL, thread_count, &a1);
pthread_create(&t2, NULL, thread_count, &a2);

pthread_join(t1, NULL);
pthread_join(t2, NULL);
long total_multi = a1.count + a2.count;
clock_t end_multi = clock();
double time_multi = (double)(end_multi - start_multi) / CLOCKS_PER_SEC >

// --- Tampilkan hasil ke layar ---
printf("\n=== HASIL PERHITUNGAN ===\n");
printf("Total kata (single-thread) : %ld\n", total_single);
printf("Waktu eksekusi (single) : %.3f ms\n", time_single);
printf("Total kata (multi-thread) : %ld\n", total_multi);
printf("Waktu eksekusi (multi) : %.3f ms\n", time_multi);
printf("Percepatan: %.2fx lebih cepat\n", time_single / time_multi);

// --- Simpan hasil ke output.txt ---
FILE *out = fopen("output.txt", "w");
if (out) {
    fprintf(out, "=== HASIL PERHITUNGAN FILE: %s ===\n", argv[1]);
    fprintf(out, "Total kata (single-thread) : %ld\n", total_single);
    fprintf(out, "Waktu eksekusi (single) : %.3f ms\n", time_single);
    fprintf(out, "Total kata (multi-thread) : %ld\n", total_multi);
    fprintf(out, "Waktu eksekusi (multi) : %.3f ms\n", time_multi);
    fprintf(out, "Percepatan: %.2fx lebih cepat\n", time_single / time_multi);
    fclose(out);
    printf("\nHasil juga disimpan ke: output.txt ✓\n");
} else {
    printf("Gagal menulis ke file output.txt ✗\n");
}

free(text);
return 0;
}

```

Program ini menghitung jumlah kata dalam sebuah file teks, dan membandingkan single-thread vs multi-thread.

Cara kerjanya:

1. Program minta user menyertakan nama file teks sebagai argumen.
2. File dibuka dan isinya dibaca semua ke memory (text).
3. Single-thread:
 - o Menghitung semua kata dari awal sampai akhir file menggunakan satu fungsi.
 - o Waktu eksekusi dicatat.
4. Multi-thread:
 - o File dibagi dua bagian.
 - o Dua thread dijalankan bersamaan, masing-masing menghitung jumlah kata di bagian berbeda.
 - o Setelah selesai, hasil kedua thread dijumlahkan.
 - o Waktu eksekusi dicatat.
5. Program menampilkan:
 - o Total kata (single-thread dan multi-thread)

- o Waktu eksekusi masing-masing
- o Percepatan yang didapat dengan multi-thread dibanding single-thread

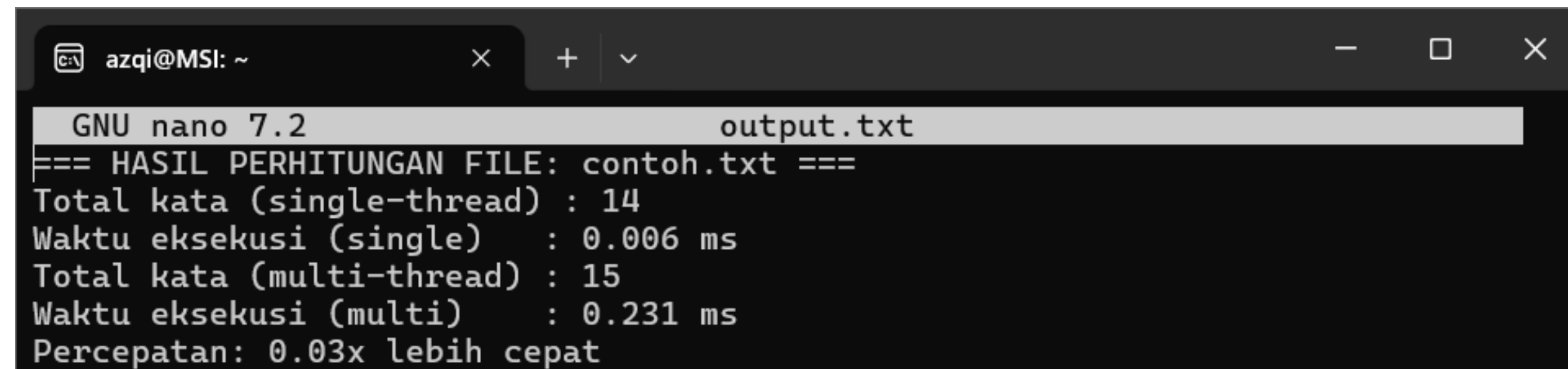
Intinya: Program ini menunjukkan bagaimana menggunakan thread untuk mempercepat proses perhitungan pada file besar.

OUTPUT:

```
azqi@MSI:~$ ./file_processing contoh.txt
```

```
=== HASIL PERHITUNGAN ===  
Total kata (single-thread) : 14  
Waktu eksekusi (single)    : 0.006 ms  
Total kata (multi-thread) : 15  
Waktu eksekusi (multi)     : 0.231 ms  
Percepatan: 0.03x lebih cepat
```

```
Hasil juga disimpan ke: output.txt ✓
```



```
GNU nano 7.2 output.txt  
=== HASIL PERHITUNGAN FILE: contoh.txt ===  
Total kata (single-thread) : 14  
Waktu eksekusi (single)    : 0.006 ms  
Total kata (multi-thread) : 15  
Waktu eksekusi (multi)     : 0.231 ms  
Percepatan: 0.03x lebih cepat
```

Tugas 3: Producer-Consumer


```
azqi@MSI: ~  
GNU nano 7.2 producer_consumer.c  
// producer_consumer.c  
// Compile: gcc -pthread -o producer_consumer producer_consumer.c  
// Jalankan: ./producer_consumer  
  
#include <stdio.h>  
#include <stdlib.h>  
#include <pthread.h>  
#include <unistd.h>  
  
#define BUFFER_SIZE 10  
  
int buffer[BUFFER_SIZE];  
int count = 0; // jumlah item dalam buffer  
int in = 0;    // indeks untuk producer  
int out = 0;   // indeks untuk consumer  
  
pthread_mutex_t mutex;  
pthread_cond_t not_full;  
pthread_cond_t not_empty;  
  
// Fungsi menghasilkan angka acak dan menambah ke buffer  
void *producer(void *arg) {  
    while (1) {  
        int item = rand() % 100; // angka random 0-99  
        pthread_mutex_lock(&mutex);  
  
        // Jika buffer penuh, tunggu  
        while (count == BUFFER_SIZE) {  
            printf("Buffer penuh, producer menunggu...\n");  
            pthread_cond_wait(&not_full, &mutex);  
        }  
  
        // Tambahkan item ke buffer  
        buffer[in] = item;  
        in = (in + 1) % BUFFER_SIZE;  
        count++;  
  
        printf("Producer menghasilkan: %d (isi buffer: %d)\n", item, count);  
    }  
}
```

```

        // Sinyalkan bahwa buffer tidak kosong
        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&mutex);

        sleep(1); // jeda agar mudah dilihat
    }
    return NULL;
}

// Fungsi mengambil item dari buffer dan "memproses"
void *consumer(void *arg) {
    while (1) {
        pthread_mutex_lock(&mutex);

        // Jika buffer kosong, tunggu
        while (count == 0) {
            printf("Buffer kosong, consumer menunggu...\n");
            pthread_cond_wait(&not_empty, &mutex);
        }

        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;

        printf("Consumer memproses: %d (isi buffer: %d)\n", item, count);

        // Sinyalkan bahwa buffer tidak penuh
        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&mutex);

        sleep(2); // waktu proses lebih lama dari producer
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;

    // Inisialisasi mutex dan kondisi

```

```

    // Inisialisasi mutex dan kondisi
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);

    // Buat thread producer dan consumer
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    // Jalankan terus (simulasi berjalan tanpa henti)
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    // Hancurkan mutex dan kondisi (tidak tercapai di loop tak hingga)
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&not_full);
    pthread_cond_destroy(&not_empty);

    return 0;
}

```

Program ini adalah simulasi Producer-Consumer menggunakan thread, mutex, dan condition variable.

Cara kerjanya:

- 1. Ada buffer dengan kapasitas terbatas (BUFFER_SIZE = 10).**
- 2. Producer (penghasil data):**
 - **Membuat angka acak 0–99.**
 - **Menambahkan angka ke buffer jika masih ada ruang.**
 - **Kalau buffer penuh → tunggu sampai consumer mengambil item.**
- 3. Consumer (pengolah data):**
 - **Mengambil item dari buffer untuk diproses.**
 - **Kalau buffer kosong → tunggu sampai producer menambahkan item.**
- 4. Mutex digunakan untuk mengamankan akses ke buffer agar tidak terjadi konflik saat producer dan consumer berjalan bersamaan.**
- 5. Condition variable (not_full & not_empty) digunakan untuk memberitahu thread lain kapan buffer penuh atau kosong, supaya mereka bisa menunggu atau melanjutkan.**
- 6. Program berjalan selamanya, menampilkan isi buffer setiap saat.**

Intinya: Program ini menunjukkan cara sinkronisasi thread untuk berbagi data di buffer tanpa bentrok, sesuai konsep klasik Producer-Consumer.

OUTPUT:

```
azqi@MSI:~$ ./producer_consumer
Producer menghasilkan: 83 (isi buffer: 1)
Consumer memproses: 83 (isi buffer: 0)
Producer menghasilkan: 86 (isi buffer: 1)
Consumer memproses: 86 (isi buffer: 0)
Producer menghasilkan: 77 (isi buffer: 1)
Producer menghasilkan: 15 (isi buffer: 2)
Consumer memproses: 77 (isi buffer: 1)
Producer menghasilkan: 93 (isi buffer: 2)
Producer menghasilkan: 35 (isi buffer: 3)
Consumer memproses: 15 (isi buffer: 2)
Producer menghasilkan: 86 (isi buffer: 3)
Producer menghasilkan: 92 (isi buffer: 4)
Consumer memproses: 93 (isi buffer: 3)
Producer menghasilkan: 49 (isi buffer: 4)
Producer menghasilkan: 21 (isi buffer: 5)
Consumer memproses: 35 (isi buffer: 4)
Producer menghasilkan: 62 (isi buffer: 5)
Producer menghasilkan: 27 (isi buffer: 6)
Consumer memproses: 86 (isi buffer: 5)
Producer menghasilkan: 90 (isi buffer: 6)
Producer menghasilkan: 59 (isi buffer: 7)
Consumer memproses: 92 (isi buffer: 6)
Producer menghasilkan: 63 (isi buffer: 7)
Producer menghasilkan: 26 (isi buffer: 8)
Consumer memproses: 49 (isi buffer: 7)
Producer menghasilkan: 40 (isi buffer: 8)
Producer menghasilkan: 26 (isi buffer: 9)
Consumer memproses: 21 (isi buffer: 8)
Producer menghasilkan: 72 (isi buffer: 9)
Producer menghasilkan: 36 (isi buffer: 10)
Consumer memproses: 62 (isi buffer: 9)
Producer menghasilkan: 11 (isi buffer: 10)
Buffer penuh, producer menunggu...
Consumer memproses: 27 (isi buffer: 9)
Producer menghasilkan: 68 (isi buffer: 10)
Buffer penuh, producer menunggu...
Consumer memproses: 90 (isi buffer: 9)
Producer menghasilkan: 67 (isi buffer: 10)
Buffer penuh, producer menunggu...
Consumer memproses: 59 (isi buffer: 9)
Producer menghasilkan: 29 (isi buffer: 10)
```

